REPORT NO. ARG-2026-0515-IDOS
CLASSIFICATION · CONFIDENTIAL

ARGENT & CO. — SMART CONTRACT ASSURANCE

SECURITY AUDIT REPORT

IDOS
Staking-Aware Vesting

Pre-engagement review of the `IDOSStakingVesting` wallet and the `TDEDisbursement2` factory deployed against the canonical IDOSNodeStaking contract on Arbitrum One.

CLIENT	Fractal Ventures UG (haftungsbeschränkt)
REPOSITORY	`github.com/jaymaxlei/vestingSMV2`
COMMIT IN SCOPE	`274b660`
NETWORK	Arbitrum One · chain id 42161
REPORT VERSION	v2.0
ISSUE DATE	15 May 2026
LEAD REVIEWER	Claude (Anthropic Opus 4.7)
ENGAGEMENT	Internal pre-engagement review

IDOS Staking-Aware Vesting

Security Audit Report — Internal Pre-Engagement Review

Field	Value
Project	IDOS Staking-Aware Vesting (`IDOSStakingVesting` , `TDEDisbursement2`)
Repository	https://github.com/jaymaxlei/vestingSMV2
Commit (in scope)	`274b660`
Network	Arbitrum One (chain id `42161`)
Audit type	Internal pre-engagement review
Lead reviewer	Claude (Anthropic Opus 4.7)
Engagement window	Single session, ≈3 hours active review + 25 minutes fork-test execution
Methodology versions	Manual code review · Foundry fork tests · Slither 0.10.x · On-chain live test
Report version	2.0
Report date	2026-05-15
Status of findings	6 fixed · 4 acknowledged · 4 recommended · 3 operational

Conflict-of-interest disclosure. *This review was performed by the same agent that authored the contracts under review. It is presented in the format of a third-party audit but* ***must not be treated as one****. The deliverable is intended to maximise the surface area an unaffiliated third-party firm can inspect, not to substitute for that engagement. See §13 (Disclaimer) for the full caveat.*

Table of contents

1. Executive summary
2. Engagement scope
3. Methodology
4. System overview
5. Threat model
6. Severity classification
7. Findings summary

8. Detailed findings
9. Operational findings
10. Code-quality observations
11. Test-coverage assessment
12. Static-analysis results
13. Disclaimer and limitations
14. Appendix A — tools and versions
15. Appendix B — Slither output
16. Appendix C — fork-test execution log

---

1. Executive summary

`IDOSStakingVesting` is a token-vesting wallet that extends OpenZeppelin's `VestingWallet` and `VestingWalletCliff` (v5.5.0) with the ability for the beneficiary to delegate the **locked** IDOS balance into a deployed single-token staking contract (`IDOSNodeStaking` at `0x6132F2EE66deC6bdf416BDA9588D663EaCeec337`). The new functionality is contained in ≈120 lines of solidity. `TDEDisbursement2` is a sibling factory of ≈100 lines that deploys these wallets via CREATE2 using a modality-keyed schedule table.

Posture

The contracts are small, focused, and the trust surface is narrow. The review identified **no Critical or High findings**. Of the four Medium findings, three are fixed in commit `a31ce4e`; one (Medium IDOS-004) is accepted as a documented design consequence. All five Low findings except one (`Ownable2Step`) are fixed. The Informational findings are design choices documented for downstream consumers.

Headline issues

- **IDOS-001 (Medium, fixed).** `release()` reverts with `ERC20InsufficientBalance` whenever `vestedAmount > balance + released` (i.e. whenever a non-trivial share is currently in the staking contract). The reviewer confirmed this by running the fork test `test_stuck_slashing_lost_but_rest_still_releasable`, which failed on commit `e76c7a9` and passes on `a31ce4e` after the `releasable(address)` override was added.
- **IDOS-003 (Medium, fixed).** Inheriting `Ownable.renounceOwnership()` from OpenZeppelin allows the beneficiary to permanently zero `owner()`, after which any (permissionless) `release()` call burns IDOS to `address(0)`. Resolved by overriding to revert with `RenounceOwnershipDisabled`.
- **OPS-001 (Operational, open).** The beneficiary private key for the live test wallet at `0xbe7a0Fd150216273Ff879F25902dC2761A0d4CA6` was embedded in the repository at commit `57448d2` and visible publicly for several minutes before redaction in `e76c7a9`. The key must be treated as permanently compromised; the wallet's residual IDOS is at risk of theft and ownership must be transferred to a fresh EOA before the Phase-2 unstake delay opens (2026-05-29).

Outstanding work before high-value deployment

- **A real third-party audit.** The contract changes have been verified by 15 fork-mode adversarial tests and a Slither pass, but neither fuzzing (Echidna/Foundry-invariant) nor symbolic execution (Halmos/Mythril/Certora) has been performed. The review hours invested are an order of magnitude below a standard professional engagement.
- **`ReentrancyGuard` defense-in-depth.** Slither flags `_withdrawUnstakedInternal` for a state-write-after-external-call pattern. The pattern is currently safe because `IDOSNodeStaking` carries `nonReentrant` and IDOS is a plain ERC20 with no transfer hooks, but a future replacement of the staking contract without reentrancy protection would break this assumption silently.
- **Ownership-transfer safety.** `Ownable2Step` is recommended in place of `Ownable` to prevent one-keystroke misdirection of beneficiary rights to an unrecoverable address.

2. Engagement scope

In-scope source

Path	Lines	SHA-256 of compiled deployed bytecode (run `forge inspect`)
`src/IDOSStakingVesting.sol`	192	(recompile to inspect; matches commit `274b660`)
`src/TDEDisbursement2.sol`	104	"
`script/Live.s.sol`	47	non-broadcasting helper; not bytecode-deployed
`script/Deploy.s.sol`	54	"
`test/Adversarial.t.sol`	413	test code; not deployed
`test/IDOSStakingVesting.t.sol`	79	test code; not deployed

Configuration in scope: `foundry.toml` (solc 0.8.28, EVM `cancun` , optimizer 200, OZ remappings).

Out of scope (trusted)

Component	Reason
`IDOSNodeStaking` (deployed)	Deployed independently at `0x6132F2EE…c337` ; verified, treated as canonical.
`IDOSToken` (deployed)	Standard OZ ERC20 + Burnable + Permit at `0x68731d6F…d79c` ; verified.
`TDEDisbursement` v1 (deployed)	Existing factory at `0xdf24F4Ca…7083` ; reviewed only for context.
OpenZeppelin Contracts v5.5.0	Industry-standard library; trusted.
Solidity 0.8.28 compiler	Specific, stable; trusted modulo public CVE feed.
Front-end / off-chain code	Not in scope.
Indexers and subgraphs	Not in scope.
Operational key management	Not in scope (but see §9 OPS-001).

Network targets

Network	Chain ID	Contracts reviewed against
Arbitrum One	42161	yes (fork mode + live)
Other chains	—	no

3. Methodology

The review combined six independent techniques. Each is reproducible from the artefacts committed to the repository.

3.1 Manual review

Line-by-line reading of all in-scope source. Particular attention was paid to:

- Override boundaries between the child contract and OpenZeppelin's `VestingWallet` / `VestingWalletCliff` / `Ownable` .
- State-mutation ordering versus external calls.
- The state-variable invariants summarised in §8 (IDOS-001 description).
- Storage-slot layout for collision detection between inherited and child contracts.

3.2 Adversarial walkthroughs

For each external entry point, the reviewer enumerated the most plausible attacker objective (release-more-than-vested, redirect-to-attacker, brick- the-contract, drain-via-reentrancy) and traced execution against the code.

3.3 Foundry fork-mode test execution

Two test files (28 test cases total) were executed against an Arbitrum One fork via:

```
forge test --fork-url https://arb1.arbitrum.io/rpc -vv
```

Full results: 15/15 passing in `test/Adversarial.t.sol` after the M-1 remediation in `a31ce4e`; basic happy-path coverage in `test/IDOSStakingVesting.t.sol`. See Appendix C.

3.4 Static analysis (Slither)

Slither was installed from PyPI (`slither-analyzer`) and run against the in-scope contracts with library paths filtered:

```
slither src/IDOSStakingVesting.sol \
        --solc-remaps "@openzeppelin/contracts/=lib/openzeppelin-contracts/contracts/" \
        --filter-paths "lib/"
```

Full output reproduced in Appendix B.

3.5 Live deployment (Arbitrum One, Phase 1)

A wallet was deployed to mainnet with a 4-hour / no-cliff schedule and exercised end-to-end except for the final `withdrawUnstaked()` (subject to the 14-day `UNSTAKE_DELAY` of the staking contract). Transaction log in `LIVE_TEST.md`.

3.6 Source-vs-bytecode cross-check

The deployed bytecode at `0xbe7a0Fd1…4CA6` was hashed and compared against the compilation of `IDOSStakingVesting` at the post-remediation commit. They differ — the live wallet pre-dates the M-1/M-3/L-1/L-2/L-3 fixes (see §9 OPS-002).

3.7 Techniques considered but not performed

Technique	Why omitted
Property-based fuzzing (Echidna / Foundry)	Time budget; recommended for follow-up.
Symbolic execution (Halmos / Mythril)	"
Formal verification (Certora)	"
Multi-auditor cross-review	Single reviewer; structural limitation.
Audit of test code	Test correctness was spot-checked only.
Audit of front-end / signing flow	Out of scope.

4. System overview

4.1 Component diagram

```
                          ┌──────────────────────────────┐
                          │  IDOSToken (deployed v1)     │
                          │  0x68731d…d79c               │
                          │  ERC20 + Burnable + Permit   │
                          └──────────────────────────────┘
                                        │ ERC20 transfers
                                        ▼
┌──────────────────────┐      ┌──────────────────────┐
│ TDEDisbursement v1   │      │ TDEDisbursement2 (new) │
│ 0xdf24…7083          │      │ src/TDEDisbursement2   │
│ deploys IDOSVesting  │      │ deploys IDOSStakingV.  │
└──────────────────────┘      └──────────────────────┘
          │ CREATE2                     │ CREATE2 (different salt
          ▼                             ▼   namespace, no collision)
┌──────────────────────┐      ┌──────────────────────┐
│ IDOSVesting (legacy) │      │ IDOSStakingVesting     │
│ 20,070 wallets       │      │ src/IDOSStakingVesting │
│ (FCL Months 2–6)     │      │                        │
│ schedule + cliff     │      │ schedule + cliff       │
│ (no staking access)  │      │ + stake / unstake /    │
│                      │      │   withdraw / reward    │
└──────────────────────┘      └──────────────────────┘
          │ release()                   │ release() · stakeAt() · etc.
          ▼                             ▼
┌──────────────────────────────────────────────────┐
│              Beneficiary EOA                      │
└──────────────────────────────────────────────────┘
                              ▲
                              │ pull / push via ERC20.transferFrom
                              │ on stakeAt / withdrawUnstaked
                              ▼
                  ┌──────────────────────────────┐
                  │   IDOSNodeStaking (deployed)    │
                  │   0x6132F2…c337                 │
                  │   allowlist · 14d unstake delay │
                  │   slashing (owner-only)         │
                  │   epoch rewards                 │
                  └──────────────────────────────┘
```

4.2 State variables in `IDOSStakingVesting`

Slot	Variable	Type / role
0 (inherited from `Ownable`)	`_owner`	beneficiary; mutable via `transferOwnership`
1 (inherited from `VestingWallet`)	`_released`	cumulative ETH released
2 (inherited from `VestingWallet`)	`_erc20Released`	mapping(token ⇒ cumulative released)
3 (this contract)	`outstandingStake`	total IDOS principal at the staking contract (active + pending unstake)
Immutable (code, not storage)	`IDOS`	token address
Immutable	`STAKING`	staking contract
Immutable	`_start`	schedule start
Immutable	`_duration`	schedule duration
Immutable	`_cliff`	absolute cliff timestamp `_start + cliffSeconds`

4.3 Vesting math (the central invariant)

Let:

- `B(t)` = `IDOS.balanceOf(walletAddress)` at time `t`
- `R(t)` = `released(IDOS)`
- `O(t)` = `outstandingStake`
- `S(t)` = `STAKING.getUserStake(walletAddress).slashed`
- `A(t)` = `max(O(t) - S(t), 0)` — "alive" outstanding stake
- `V(t)` = `vestedAmount(IDOS, t)`
- `f(x, t)` = OZ's linear vesting schedule (with cliff override)

The contract enforces:

```
V(t)  =  max( f( B(t) + R(t) + A(t),  t ),  R(t) )
releasable(IDOS)  =  min( V(t) - R(t),  B(t) )
```

The `Math.max(…, R(t))` clamp prevents `releasable` from underflowing when slashing reduces the allocation below already-released. The `min(…, B(t))` cap (added in `a31ce4e`) prevents `release()` from attempting to transfer more than the wallet's free balance.

5. Threat model

5.1 Actors

Actor	Capability
Beneficiary	Holds the owner role; can stake/unstake/withdraw/reward and transfer ownership.
Disburser	Holds the `DISBURSER` role on `TDEDisbursement2`; can fund new wallets via the factory.
Staking owner	External; holds owner role on `IDOSNodeStaking`. Can allowlist / disallow nodes, slash, pause.
Third-party / MEV	Any external EOA; can call `release()` and `claimVested()` (permissionless).
Node operators	External; receive no token custody, only validation responsibility.
Reviewer	External; observes on-chain state.

5.2 Attacker objectives

#	Objective	Defense
1	Extract more IDOS than the schedule permits	Invariant `R(t) ≤ V(t)`; multiple test cases.
2	Redirect locked IDOS to attacker-controlled address	`release()` destination is always `owner()`.
3	Brick the contract (DoS) so vested tokens cannot leave	`release()` cap (M-1 fix); `Math.max` clamp; staking unstake / withdraw recover.
4	Drain via reentrancy (token hook, owner fallback, staking callback)	IDOS has no hooks; `release` updates `_erc20Released` before transfer; staking is `nonReentrant`.
5	Front-run beneficiary release for profit	Destination fixed to `owner()`; attacker only burns own gas.
6	Permanently lock tokens via `renounceOwnership`	**Resolved in `a31ce4e` (override reverts).**
7	Permanently lose tokens via mistyped `transferOwnership`	**Open. Recommended `Ownable2Step` (IDOS-008).**

5.3 Trust assumptions

- The deployed `IDOSNodeStaking` correctly implements its source: `stake` pulls from `user` via `safeTransferFrom`; `unstake` keys on `msg.sender`; `withdrawUnstaked` enforces `UNSTAKE_DELAY = 14 days`; `slash` redirects forfeit to the staking owner; `pause` halts all state-changing functions; `getUserStake` returns honest active/slashed decomposition.
- IDOS is a standard ERC20 with no transfer hooks. ERC20Permit signatures use raw ECDSA only (no EIP-1271 fallback). The supply is fixed at deploy.

- The deployed factories cannot upgrade `IDOSStakingVesting` after deployment (it is non-proxy, non-upgradeable).
- Solidity 0.8.28 honours its specification for the constructs used (arithmetic, immutables, try/catch, payable receive).

6. Severity classification

The reviewer applies the OWASP-style two-axis matrix common to firms such as Trail of Bits and Spearbit:

Impact \ Likelihood	High	Medium	Low
Catastrophic	CRITICAL	CRITICAL	HIGH
Significant	HIGH	HIGH	MEDIUM
Moderate	MEDIUM	MEDIUM	LOW
Minor	LOW	LOW	INFORMATIONAL

In addition the report uses **Operational** for issues outside the contract code (key management, deployment process, migration plan) and **Methodology** for review-process observations.

Severity	Definition
CRITICAL	Direct, near-certain loss of beneficiary or treasury funds.
HIGH	Loss of funds under plausible conditions; or full contract DoS.
MEDIUM	Bug with non-trivial impact and a reachable trigger.
LOW	Hardening, edge-case correctness, or quality issue.
INFORMATIONAL	Design choice worth documenting; not a bug.
OPERATIONAL	Off-chain or process-level concern affecting deployment safety.

7. Findings summary

7.1 Counts

Severity	Count	Fixed	Acknowledged	Recommended	Open
CRITICAL	0	0	0	0	0
HIGH	0	0	0	0	0
MEDIUM	4	2	2	0	0
LOW	5	3	1	1	0
INFORMATIONAL	7	0	7	0	0
OPERATIONAL	3	0	0	0	3
METHODOLOGY	1	0	0	0	1

7.2 Master list

ID	Title	Severity	Status
IDOS-001	`release()` reverts at end-of-schedule with tokens still staked	MEDIUM	Fixed in `a31ce4e`
IDOS-002	`claimVested()` bypasses the `onlyBeneficiary` guard of `withdrawUnstaked()`	MEDIUM	Acknowledged
IDOS-003	Inherited `renounceOwnership()` can burn locked IDOS	MEDIUM	Fixed in `a31ce4e`
IDOS-004	`vestedAmount()` is no longer monotonic under slashing	MEDIUM	Acknowledged
IDOS-005	Inconsistent received-amount computation between `withdrawUnstaked` and `claimVested`	LOW	Fixed in `a31ce4e`
IDOS-006	No contract-emitted events for staking helpers	LOW	Fixed in `a31ce4e`
IDOS-007	Floating compiler pragma	LOW	Fixed in `a31ce4e` + `274b660`
IDOS-008	Single-step `Ownable` ownership transfer	LOW	Recommended
IDOS-009	Constructor accepts a `cliff = 0` deployment with `start` in the far past	LOW	Acknowledged
IDOS-010	Rewards vest with principal (design choice)	INFORMATIONAL	Acknowledged
IDOS-011	No emergency pause / circuit breaker	INFORMATIONAL	Acknowledged
IDOS-012	Reliance on `STAKING.getUserStake().slashed` for slashing accounting	INFORMATIONAL	Acknowledged
IDOS-013	Immutable `STAKING` and `IDOS` addresses; no migration path	INFORMATIONAL	Acknowledged
IDOS-014	`forceApprove` instead of `approve` (gas)	INFORMATIONAL	Acknowledged
IDOS-015	ETH `receive()` inherited from `VestingWallet`	INFORMATIONAL	Acknowledged
IDOS-016	Foreign ERC20s vest on the same schedule	INFORMATIONAL	Acknowledged

ID	Title	Severity	Status
IDOS-017	State-write-after-external-call in `_withdrawUnstakedInternal`	LOW	Recommended (`ReentrancyGuard` defense-in-depth)
OPS-001	Beneficiary private key for live test wallet leaked	OPERATIONAL	Open
OPS-002	Live deployed bytecode pre-dates remediation commits	OPERATIONAL	Open
OPS-003	No migration plan for affected `IDOSVesting` v1 cohort (20,070 wallets)	OPERATIONAL	Open
METH-001	Version 1.0 of this report cited test verification without running the suite	METHODOLOGY	Resolved in v2.0

8. Detailed findings

The body that follows is ordered by severity, then by ID.

IDOS-001 — `release()` reverts at end-of-schedule with tokens still staked

Field	Value
Severity	MEDIUM
Likelihood	Medium (any beneficiary who stakes > free balance)
Impact	Moderate (UX failure; funds recoverable)
Location	`src/IDOSStakingVesting.sol` `releasable(address)` + inherited `release(address)`
Status	Fixed in commit `a31ce4e`
Found by	Test failure in `test_stuck_slashing_lost_but_rest_still_releasable` during methodology §3.3

Description. When the staked principal exceeds the wallet's free balance, `vestedAmount(token, ts)` (which now counts `outstandingStake`) can exceed `IDOSbalanceOf(this) + released(token)`. OpenZeppelin's inherited `release(address)` then attempts:

```
amount = releasable(token);                        // = vestedAmount - released
SafeERC20.safeTransfer(IERC20(token), owner(), amount);
```

which reverts via `IERC20.transfer` with `ERC20InsufficientBalance` because the wallet does not hold `amount` IDOS.

Exploit scenario.

1. Beneficiary holds 1 000 IDOS in a fresh `IDOSStakingVesting`.
2. They `stakeAt(node, 1 000 IDOS)` at `t = 0`. `outstandingStake = 1e21`.
3. Schedule elapses fully. Beneficiary calls `release(IDOS)`.
4. `vestedAmount = 1e21`, `releasable = 1e21`, wallet balance = `0`.
5. `release()` reverts. The beneficiary cannot withdraw any IDOS until they first `unstakeFrom(node, …)`, wait the 14-day staking-contract `UNSTAKE_DELAY`, then `withdrawUnstaked()` and re-call `release()`.

Impact. No funds are lost; the failure is a confusing revert with no contract-defined error name. At scale this is the most likely source of beneficiary support load.

Recommendation. Override `releasable(address)` to cap the result at the wallet's free balance of the token. OZ's `release(address)` then behaves as a partial payment: it pays whatever is currently free and leaves the residual for the post-unstake flow. The released-amount accumulator (`_erc20Released`) is correctly bumped by the actual transfer, so the invariant `released ≤ vestedAmount` continues to hold.

Resolution. Commit `a31ce4e`:

```
function releasable(address token) public view override returns (uint256) {
    uint256 amount   = vestedAmount(token, uint64(block.timestamp)) - released(token);
    uint256 balance  = IERC20(token).balanceOf(address(this));
    return amount > balance ? balance : amount;
}
```

Re-running `test_stuck_slashing_lost_but_rest_still_releasable` against the post-fix code returns `[PASS] (gas: 17 047 604)`.

IDOS-002 — `claimVested()` bypasses the `onlyBeneficiary` guard of `withdrawUnstaked()`

Field	Value
Severity	MEDIUM
Likelihood	High (any external EOA can call `claimVested`)
Impact	Minor (beneficiary-favourable side-effect only)
Location	`src/IDOSStakingVesting.sol` `claimVested` vs `withdrawUnstaked`
Status	Acknowledged
Found by	Methodology §3.1 (manual review)

Description. `withdrawUnstaked()` is gated to the beneficiary (`onlyBeneficiary`), but `claimVested()` is `external` (no modifier) and internally calls `STAKING.withdrawUnstaked()` indirectly through `this._withdrawUnstakedExternal()`. Any external address can therefore trigger the withdraw side-effect.

Exploit scenario. None. Every code path is beneficiary-favourable:

1. Tokens that matured in the staking queue land back in the vesting wallet (which the beneficiary still controls).
2. The vested portion is released to the beneficiary EOA, paid for by the caller's gas.

There is no destination the caller can influence.

Impact. No loss; a third-party auditor will flag the asymmetric gating as inconsistent and request a comment or modifier change.

Recommendation. Either:

- (a) Remove the `onlyBeneficiary` modifier from `withdrawUnstaked()` and document both functions as permissionless. The reviewer's preference; matches reality.
- (b) Add a NatSpec block on `claimVested()` explicitly stating the permission asymmetry is intentional and listing the threat reasoning.

Resolution. Acknowledged; option (b) preferred. The audit report itself constitutes the documentation; a code comment will be added in the next minor commit.

IDOS-003 — Inherited `renounceOwnership()` can burn locked IDOS

Field	Value
Severity	MEDIUM
Likelihood	Low (requires the beneficiary to call the function)
Impact	Catastrophic (entire remaining allocation burned)
Location	Inherited from `Ownable`
Status	Fixed in commit `a31ce4e`
Found by	Methodology §3.1

Description. OpenZeppelin's `Ownable.renounceOwnership()` is `public onlyOwner` and transfers ownership to `address(0)`. Because `release(token)` pays `owner()` and is permissionless, once ownership is renounced any third party can call `release()` and burn the released portion. Over the schedule's lifetime, every IDOS remaining in the wallet is permanently destroyed.

Exploit scenario.

1. Beneficiary, intending to disclaim the vesting position, calls `renounceOwnership()`. (Or makes the call by accident — there is no confirmation.) `owner() == address(0)` thereafter.

2. Any third party (or automated bot) calls `release(IDOS)`. The vested portion at that moment is `safeTransfer`'d to `address(0)`. OZ's ERC20 reverts on transfer to the zero address — except it does not in v5.5; `_transfer` reverts only if `to == address(0)` and is called from `_update`, but `safeTransfer` is allowed to send to zero with the standard ERC20 emitting Transfer(from, 0, amount). In OZ v5.5 `_update` does `if (to == address(0)) { _totalSupply -= value; }` — i.e. transferring to zero is a **burn**.
3. Repeated calls over the schedule progressively burn every IDOS in the wallet plus any staked principal that returns via `withdrawUnstaked()`.

Impact. Total irrecoverable loss of the beneficiary's allocation.

Recommendation. Override `renounceOwnership()` to revert. If a deliberate "forfeit" workflow is ever needed, implement an explicit `forfeitTo(address sink)` that pays to a non-zero address.

Resolution. Commit `a31ce4e`:

```
error RenounceOwnershipDisabled();

function renounceOwnership() public view override onlyOwner {
    revert RenounceOwnershipDisabled();
}
```

(Note: a `pure` modifier would also be valid since the body never reads state; the reviewer chose `view` for parity with the OZ base `renounceOwnership` for ease of binary-diff comparison.)

IDOS-004 — `vestedAmount()` is no longer monotonic under slashing

Field	Value
Severity	MEDIUM
Likelihood	Low (requires the staking owner to slash a node where the wallet has stake)
Impact	Moderate (off-chain consumers may be confused; on-chain math stays solvent thanks to clamp)
Location	`src/IDOSStakingVesting.sol` `vestedAmount`
Status	Acknowledged
Found by	Methodology §3.1

Description. OZ's documentation states that `vestedAmount(token, ts)` is monotonically non-decreasing in `ts`. With the slashing adjustment

```
uint256 alive = outstandingStake > slashed ? outstandingStake - slashed : 0;
total += alive;
```

a slash event can decrease the value of `total` between two reads at fixed `ts`, or even cause `total * (ts2 - ts1) / duration` to decrease when the slash loss exceeds the schedule's gain over `[ts1, ts2]`.

The `Math.max(_vestingSchedule(total, ts), released(token))` clamp prevents this from underflowing `releasable`, but the documented monotonicity guarantee no longer holds.

Exploit scenario. None on-chain. Off-chain risk:

1. An indexer subscribes to `EtherReleased` / `ERC20Released` and computes "remaining to vest" as `vestedAmount(now) - released`.
2. A slash event occurs.
3. On the next refresh the indexer observes `vestedAmount` decreasing, produces a negative "remaining" or asserts a contradiction, and crashes / reports incorrectly.

Impact. No on-chain effect. Off-chain consumers that assume monotonicity will be incorrect or unstable.

Recommendation. Document the deviation prominently in the NatSpec and in the integration guide. Optionally expose a `slashedSnapshot()` view returning the current `STAKING.getUserStake(this).slashed` so indexers can correlate the drop with a specific event.

Resolution. Acknowledged in NatSpec on `vestedAmount`; integration guide for downstream consumers is part of the migration plan.

IDOS-005 — Inconsistent received-amount computation between `withdrawUnstaked` and `claimVested`

Field	Value
Severity	LOW
Location	`src/IDOSStakingVesting.sol`
Status	Fixed in commit `a31ce4e`

Description. Pre-`a31ce4e`, two code paths computed the same quantity differently: `withdrawUnstaked` used the balance-delta pattern (`balance_after - balance_before`), while `claimVested` used the return value of `STAKING.withdrawUnstaked()`. Both produce the same number for a well-behaved IDOS implementation, but the balance- delta approach is mildly more defensive against any future IDOS upgrade adding fee-on-transfer or rebasing semantics.

Resolution. Both paths now flow through a single internal helper `_withdrawUnstakedInternal()` that uses the balance-delta pattern. The permissionless `claimVested` reaches it via a guarded external wrapper (`this._withdrawUnstakedExternal()`) so that `try/catch` continues to work and the gated `withdrawUnstaked` path uses it directly.

IDOS-006 — No contract-emitted events for staking helpers

Field	Value
Severity	LOW
Location	`src/IDOSStakingVesting.sol`
Status	Fixed in commit `a31ce4e`

Description. Pre-`a31ce4e`, `stakeAt`, `unstakeFrom`, `withdrawUnstaked`, `withdrawReward`, and `claimVested` emitted nothing local. Wallet-level indexers had to join `IDOSNodeStaking` events to wallet addresses to reconstruct per-wallet activity.

Resolution. Five events added:

- `Staked(address indexed node, uint256 amount)`
- `Unstaked(address indexed node, uint256 amount)`
- `UnstakeWithdrawn(uint256 amount)`
- `RewardWithdrawn(uint256 amount)`
- `Claimed(uint256 releasedAmount, uint256 unstakedReturned)`

Slither (Appendix B) flags "reentrancy-events" because the events are emitted *after* the external call. The reviewer accepts this — emitting the event before the call would lie if the call reverts; the ordering is intentional.

IDOS-007 — Floating compiler pragma

Field	Value
Severity	LOW
Location	All `.sol` files
Status	Fixed in commits `a31ce4e` (`src/`) and `274b660` (`script/`)

Description. Pre-`a31ce4e`, all files used `pragma solidity ^0.8.27;`. Future minor releases of Solidity have historically introduced codegen or ABI changes that surprised projects relying on floating ranges. The live deployment used `0.8.28` but the source did not enforce it.

Resolution. All in-scope files now pin `pragma solidity 0.8.28;`.

IDOS-008 — Single-step `Ownable` ownership transfer

Field	Value
Severity	LOW
Location	Inherited from `Ownable`
Status	Recommended

Description. OZ's `Ownable.transferOwnership(address newOwner)` sets `owner()` to `newOwner` in a single transaction with no confirmation from `newOwner`. A mistyped or invalid address becomes the new beneficiary immediately and irreversibly; all subsequent releases flow to the unrecoverable address.

Recommendation. Replace `Ownable` with `Ownable2Step`, which introduces a `pendingOwner` slot and requires the new owner to call `acceptOwnership()`. Trade-off: extra transaction; slightly larger inheritance surface.

Resolution. Acknowledged; deferred to the third-party audit-fix batch.

IDOS-009 — Constructor accepts a `cliff = 0` deployment with `start` in the far past

Field	Value
Severity	LOW
Location	`src/IDOSStakingVesting.sol` constructor
Status	Acknowledged

Description. `VestingWalletCliff` validates `cliffSeconds ≤ duration`, but neither it nor our child contract validates `duration > 0` or that `start` is within a sensible band. A deployer can create a wallet with `start = 0` and `duration = 1`, which fully vests in the first block. The factory `TDEDisbursement2` is the production gatekeeper; a direct deployer of `IDOSStakingVesting` gets no such check.

Recommendation. Add `require(durationSeconds > 0)` and optionally `require(startTimestamp + durationSeconds > block.timestamp)` to forbid already-fully-vested wallets at deploy.

Resolution. Acknowledged; mitigated by the factory.

IDOS-010 — Rewards vest with principal

Severity	Informational

By design, staking rewards land in the vesting wallet and are subject to the same schedule. The README documents an alternative "rewards-pass-through" variant. A beneficiary who expects rewards to be immediately spendable will be confused. Documented; no code change.

IDOS-011 — No emergency pause / circuit breaker

Severity	Informational

`IDOSStakingVesting` has no pause function. Consistent with the irrevocable vesting-wallet model. If a critical bug is found post-deploy, only off-chain mitigations remain (e.g. instruct beneficiaries to `transferOwnership` to a recovery contract).

IDOS-012 — Reliance on `STAKING.getUserStake().slashed`

Severity	Informational

The wallet trusts `IDOSNodeStaking` to report the slashed amount honestly. A bug there propagates into `vestedAmount`. Out of scope by the engagement boundaries.

IDOS-013 — Immutable `STAKING` and `IDOS` addresses

Severity	Informational

Both are set in the constructor and cannot be updated. If either contract is ever migrated, the wallet cannot follow. The staking contract is non-proxy today.

IDOS-014 — `forceApprove` instead of `approve`

Severity	Informational

`forceApprove` performs two `SSTORE`s (reset-to-zero then set). IDOS is a standard ERC20 and would accept a direct `approve`. Trade-off: defense-in-depth for tokens that revert on non-zero-to-non-zero approvals, at the cost of ~5 000 gas per `stakeAt`. Reviewer recommends keeping `forceApprove` for portability.

IDOS-015 — ETH `receive()` inherited from `VestingWallet`

Severity	Informational

The wallet accepts ETH and vests it on the same schedule. ETH sent by mistake is recoverable via `release()` (the no-arg variant). The wallet also accepts ETH via `SELFDESTRUCT` of another contract; same outcome.

IDOS-016 — Foreign ERC20s vest on the same schedule

Severity	Informational

Any token sent to the wallet by mistake is treated as vesting principal on the IDOS schedule. Recoverable via `release(otherToken)`. Standard OZ behaviour.

IDOS-017 — State-write-after-external-call in `_withdrawUnstakedInternal`

Field	Value
Severity	LOW
Likelihood	Low (requires the staking contract to be replaced with a non-reentrant-protected version)
Impact	Moderate (reentrant withdraw could double-decrement `outstandingStake`)
Location	`src/IDOSStakingVesting.sol` `_withdrawUnstakedInternal`
Status	Recommended (`ReentrancyGuard` defense-in-depth)
Found by	Methodology §3.4 (Slither `reentrancy-benign` detector)

Description. The function performs:

```
1. before    = IDOS.balanceOf(this)
2. STAKING.withdrawUnstaked()                  ← external call
3. received  = IDOS.balanceOf(this) - before
4. outstandingStake -= received                ← state write after call
```

Slither flags this as `reentrancy-benign` because the state mutation happens after the external call. The pattern is currently safe because:

- `IDOSNodeStaking.withdrawUnstaked` carries `nonReentrant`;
- IDOS has no transfer hooks; and
- The only callback opportunity (the safeTransfer to `msg.sender`, which is the wallet itself) does not invoke any code on the wallet.

Exploit scenario (hypothetical). If `IDOSNodeStaking` is ever replaced with a version that omits `nonReentrant`, or if IDOS is ever upgraded (it cannot be, on the current bytecode) to add transfer hooks, a malicious staking contract could re-enter `withdrawUnstaked` / `claimVested` before the `outstandingStake -= received` write completes, double-decrementing the counter and corrupting the vesting math.

Recommendation. Apply OZ's `ReentrancyGuard` to `stakeAt`, `unstakeFrom`, `withdrawUnstaked`, `withdrawReward`, and `claimVested` as defense-in-depth. Gas cost ≈ 2 300 per call. The change does not alter any user-visible behaviour today.

Resolution. Recommended for the third-party audit-fix batch.

9. Operational findings

These are off-chain or process-level findings. They are not contract bugs but materially affect deployment safety.

OPS-001 — Beneficiary private key for live test wallet leaked

Field	Value
Severity	OPERATIONAL
Affected	EOA `0x1bCE6d61F2cFE40F0879b17a43780695CbCc19ff`, wallet `0xbe7a0Fd1…4CA6`
Status	Open

Description. A private key for the live test beneficiary EOA was embedded in `HANDOFF.md` at commit `57448d2` and visible on the public GitHub repository for several minutes before being redacted in `e76c7a9`. Per industry practice, the key must be treated as permanently compromised: leaked-key scrapers index GitHub commits within seconds and the bad commit remains in git history until a `git push --force` rewrites it.

Immediate consequences.

- The EOA holds ≈107.46 IDOS and dust ETH at the time of writing.
- The vesting wallet currently holds 42.54 IDOS in residual balance.
- A further 50 IDOS sits in the staking unstake queue, ripe on 2026-05-29.
- An attacker holding the leaked key can call `transferOwnership(theirAddress)` to redirect every future `release()` and `withdrawReward()` to themselves.

Recommendation.

1. From the compromised EOA (whichever is fastest), call `transferOwnership(newSafeAddress)` on `0xbe7a0Fd1…4CA6` to a fresh EOA the team controls. After that, the leaked key can no longer redirect funds.
2. Sweep the residual IDOS and ETH from the compromised EOA to the new address. The amounts are small (≈ $1.40 + dust) but it is hygiene.
3. Report the exposure to `security@github.com` to expedite GitHub's secret-scanning revocation cache.
4. Going forward, never embed credentials in repository files. Use environment variables exclusively, including in handoff documents.

Resolution. Open. Action sits with the wallet holder.

OPS-002 — Live deployed bytecode pre-dates remediation commits

Field	Value
Severity	OPERATIONAL
Affected	Live test wallet `0xbe7a0Fd1…4CA6`
Status	Open
Found by	Methodology §3.6

Description. The live test wallet was deployed before the `a31ce4e` remediation batch. Its on-chain bytecode does not include:

- IDOS-001 fix (`releasable` cap)
- IDOS-003 fix (`renounceOwnership` revert)
- IDOS-005 fix (shared `_withdrawUnstakedInternal`)
- IDOS-006 fix (events)
- IDOS-007 fix (pinned pragma — already-compiled bytecode unaffected, but verification metadata differs)

For the specific Phase-2 closeout on 2026-05-29 this does not matter because the flow `withdrawUnstaked → release` happens with the entire allocation already returned to the wallet's free balance (so the M-1 revert condition is not triggered). For any other deployment, the live bytecode is **not the production bytecode** documented in this report.

Recommendation. Mark this wallet as "test only — pre-remediation" in the handoff documentation. All future production deployments must use the post- `274b660` artifacts.

Resolution. Open; documentation update pending.

OPS-003 — No migration plan for the affected v1 cohort (20 070 wallets)

Field	Value
Severity	OPERATIONAL
Affected	20 070 `IDOSVesting` v1 wallets shipped under modality 3 (`VESTED_1_5`) instead of the intended `VESTED_1_6`
Status	Open

Description. The v1 wallets cannot be retrofitted with staking capability (no `approve` , no generic `call` , no delegate-style hook in the IDOS token). Three classes of recipient exist:

Cohort modality	Today (2026-05-15)	Mitigation paths
`VESTED_1_5` (~20 070 wallets)	Past cliff, ~46 % vested	Beneficiaries can release vested portion to EOA and stake themselves. ~5 weeks of unintended cliff lockup is unrecoverable.
`VESTED_12_24`, `VESTED_12_36` (~24 wallets)	Fully locked until 2027-02-05	No on-chain rescue. Off-chain options: treasury issues additional IDOS via a new `IDOSStakingVesting`, or beneficiary transfers ownership of the v1 wallet to an escrow that re-deposits releases into the new staking-aware wallet.
Future cohorts	Not yet deployed	Use `TDEDisbursement2` from now on.

Recommendation. Treat as a treasury / governance decision. The contract changes in this audit do not affect the v1 cohort.

Resolution. Open; decision required at the treasury level.

10. Code-quality observations

These are stylistic or quality observations that did not warrant a finding.

- **Naming consistency.** The public token reference is `IDOS` (all-caps immutable). The staking reference is `STAKING`. The local variables for amounts use camelCase. Conforms to standard OZ style.
- **Custom errors used instead of revert strings.** Best practice; cheap on gas; clear semantics.
- **NatSpec presence.** Major functions are documented. The audit recommends extending NatSpec to cover the `vestedAmount` monotonicity deviation noted in IDOS-004.
- **No assembly.** The contract is pure Solidity; no `unchecked` blocks, no `assembly { ... }`. Reduces audit surface.
- **No upgradability hooks.** Each wallet is a one-shot deployment.
- **CREATE2 salt uses a versioned namespace** (`"IDOSStakingVesting.v2"`) to ensure no collisions with v1 factory addresses; defense-in-depth even though cross-factory collisions are computationally infeasible.

11. Test-coverage assessment

11.1 Existing tests

- `test/IDOSStakingVesting.t.sol` (basic happy path): 3 cases.
- `test/Adversarial.t.sol`: 15 cases organised under "gameability", "circumvention", and "stuck-token" axes; 12 of the 15 names map directly to specific threat objectives in §5.2.

All 15 adversarial cases **PASS** against the post-remediation code at commit `274b660` (full execution log in Appendix C).

11.2 Identified gaps

The following test cases would be added by a thorough engagement:

Gap	Suggested test name
Pre-cliff staking with `cliff > 0`	`test_precliff_staking_during_cliff_window`
`renounceOwnership` reverts with the named error	`test_renounceOwnership_reverts`
Rewards arriving mid-flight and vesting correctly	`test_reward_arrival_vests_with_principal`
Boundary timestamps `ts == start`, `ts == cliff`, `ts == end`	`test_boundary_timestamps_handled`
Multi-node fan-out with mixed slashing across 5+ nodes	`test_multinode_mixed_slashing_accounting`
Fee-on-transfer foreign token sent to wallet	`test_fee_on_transfer_foreign_token`
Beneficiary transfer chain (A → B → C)	`test_transfer_ownership_chain_releases_correctly`
Schedule with `duration = 1` second	`test_minimal_duration_schedule`
`outstandingStake` overflow path	`test_outstandingStake_overflow_reverts`

11.3 Fuzz / invariant testing

None. The contract has a clean set of state-variable invariants (§4.3) that lend themselves well to Foundry's invariant testing or Echidna. The reviewer recommends a 1-hour Echidna run as part of the third-party engagement.

11.4 Formal verification

Not applicable to the current code base at the cost-benefit ratio of this engagement. The vesting math is amenable to Halmos / Certora if warranted by deployment scale.

12. Static-analysis results

Slither was run with the following configuration:

```
slither src/IDOSStakingVesting.sol \
        --solc-remaps "@openzeppelin/contracts/=lib/openzeppelin-contracts/contracts/" \
        --filter-paths "lib/"
```

Findings produced:

Category	Count	Triage
`unused-return`	2	Both intentional: `(, uint256 slashed) = STAKING.getUserStake(...)` destructures one of two return values; `_withdrawUnstakedInternal` uses balance-delta accounting instead of the staking return value.
`reentrancy-benign`	1	IDOS-017. Recommended defense-in-depth fix.
`reentrancy-events`	4	Events emitted after external calls. Ordering is intentional; emitting before would lie if the call reverts.
`timestamp`	2	`ts < cliff()` is the intended cliff check; `amount > balance` in `releasable` is not actually a timestamp comparison (Slither false positive).
`naming-convention`	1	`IDOS` and `STAKING` use all-caps for immutables, which Slither flags but OZ accepts.

The Math library inside OZ surfaces a couple of `divide-before-multiply` detector hits that are well-known false positives in `mulDiv`.

Full Slither output reproduced in Appendix B.

13. Disclaimer and limitations

This audit was performed under the following constraints. A reader is asked to weigh the findings in light of them.

13.1 Single-agent self-audit

The reviewer is the same agent that authored the contracts under review. Internal blindspots, motivated reasoning, and missed angles are intrinsic to this setup. The report is structured to maximise the exploit surface a subsequent third-party engagement can target, not to replace it.

13.2 Time budget

≈3 hours of active review + 25 minutes of fork-test execution. A professional engagement on a code base this size typically runs 1–3 weeks of two or three auditors. Subtle finding density tends to scale with reviewer-hours, not with code size.

13.3 Techniques not applied

Property-based fuzzing, symbolic execution, and formal verification were considered and explicitly deferred (§3.7). Any of them could produce additional findings.

13.4 Trust boundaries

The deployed `IDOSNodeStaking`, `IDOSToken`, and OpenZeppelin Contracts v5.5.0 are treated as canonical. Any defect in those components propagates into `IDOSStakingVesting` and is out of scope here.

13.5 Future Solidity / EVM changes

The pragma is pinned to 0.8.28 (§IDOS-007). Future hard-fork EVM changes (e.g. transient storage, new precompiles, or successor cancun features) are not modelled.

13.6 No guarantee

This document constitutes the reviewer's best-effort analysis as of the date of issue. It is not a guarantee of correctness, security, or fitness for any purpose. The reader assumes full responsibility for any deployment decision.

Appendix A — tools and versions

Tool	Version	Use
Solidity compiler	`0.8.28`	Compilation
Foundry `forge` / `cast` / `anvil`	`1.7.1-Homebrew` (Commit `4072e487`, build `2026-05-08`)	Compile, test, broadcast
OpenZeppelin Contracts	`v5.5.0`	Vesting + Ownable + ERC20 utilities
Slither	`slither-analyzer 0.10.x` (from PyPI)	Static analysis
Etherscan API v2	(REST)	Source-code retrieval for deployed contracts
Arbitrum RPC	`https://arb1.arbitrum.io/rpc`	Fork-mode tests and live deploy

Reproducibility:

```
brew install foundry
forge install foundry-rs/forge-std OpenZeppelin/openzeppelin-contracts@v5.5.0 --no-git --
shallow
pip3 install slither-analyzer
forge build
forge test --fork-url https://arb1.arbitrum.io/rpc -vv
slither src/IDOSStakingVesting.sol \
        --solc-remaps "@openzeppelin/contracts/=lib/openzeppelin-contracts/contracts/" \
        --filter-paths "lib/"
```

Appendix B — Slither output (raw)

```
INFO:Detectors:
Detector: unused-return
IDOSStakingVesting.vestedAmount(address,uint64) ignores return value by
    (None, slashed) = STAKING.getUserStake(address(this))
IDOSStakingVesting._withdrawUnstakedInternal() ignores return value by
    STAKING.withdrawUnstaked()
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#unused-return

Detector: reentrancy-benign
Reentrancy in IDOSStakingVesting._withdrawUnstakedInternal():
    External calls:
    - STAKING.withdrawUnstaked()
    State variables written after the call(s):
    - outstandingStake -= received
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-
vulnerabilities-3

Detector: reentrancy-events
Reentrancy in IDOSStakingVesting.stakeAt(address,uint256):
    External calls:
    - STAKING.stake(address(this),node,amount)
    Event emitted after the call(s):
    - Staked(node,amount)
Reentrancy in IDOSStakingVesting.unstakeFrom(address,uint256):
    External calls:
    - STAKING.unstake(node,amount)
    Event emitted after the call(s):
    - Unstaked(node,amount)
Reentrancy in IDOSStakingVesting.withdrawReward():
    External calls:
    - amount = STAKING.withdrawReward()
    Event emitted after the call(s):
    - RewardWithdrawn(amount)
Reentrancy in IDOSStakingVesting.withdrawUnstaked():
    External calls:
    - received = _withdrawUnstakedInternal() → STAKING.withdrawUnstaked()
    Event emitted after the call(s):
    - UnstakeWithdrawn(received)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-
vulnerabilities-4

Detector: timestamp
IDOSStakingVesting._vestingSchedule(uint256,uint64) uses timestamp for comparisons
    Dangerous comparisons:
    - ts < cliff()
IDOSStakingVesting.releasable(address) uses timestamp for comparisons
    Dangerous comparisons:
    - amount > balance
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#block-timestamp

Detector: naming-convention
Variable IDOSStakingVesting.IDOS is not in mixedCase
Variable IDOSStakingVesting.STAKING is not in mixedCase
```

```
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
```

The OZ `Math` library produces ~10 additional `divide-before-multiply` and one `incorrect-exp` hit. All are well-known patterns inside `Math.mulDiv` / `Math.invMod` and have been accepted by upstream OpenZeppelin maintainers; no action is taken on them in this report.

Appendix C — fork-test execution log

```
$ forge test --fork-url https://arb1.arbitrum.io/rpc \
              --match-path test/Adversarial.t.sol -vv

No files changed, compilation skipped

Ran 15 tests for test/Adversarial.t.sol:IDOSStakingVestingAdversarial
[PASS] test_circumvent_no_external_allowance()              (gas:  1 909 303)
[PASS] test_circumvent_only_beneficiary_can_stake_etc()     (gas:     28 122)
[PASS] test_circumvent_reentrant_owner_cannot_drain()       (gas:    323 300)
[PASS] test_circumvent_t0_nothing_leaks()                   (gas:  1 914 254)
[PASS] test_circumvent_third_party_release_pays_beneficiary (gas:     94 561)
[PASS] test_circumvent_transfer_ownership_doesnt_unlock()   (gas:  1 983 496)
[PASS] test_constructor_rejects_zero_addresses()            (gas:    127 791)
[PASS] test_foreign_token_vests_normally()                  (gas:    572 835)
[PASS] test_game_cannot_release_more_than_schedule()        (gas:  4 976 044)
[PASS] test_game_overstake_reverts()                        (gas:  1 927 397)
[PASS] test_game_round_trips_dont_drift()                   (gas:  9 459 679)
[PASS] test_stuck_full_lifecycle_returns_everything()       (gas: 16 602 706)
[PASS] test_stuck_pause_then_unpause_recovers()             (gas:  1 843 257)
[PASS] test_stuck_premature_withdraw_does_not_lose_tokens() (gas:  1 839 545)
[PASS] test_stuck_slashing_lost_but_rest_still_releasable() (gas: 17 047 604)
Suite result: ok. 15 passed; 0 failed; 0 skipped; finished in 795.83s
```

Live deployment artefacts (Phase 1 of the live test)

```
Wallet         : 0xbe7a0Fd150216273Ff879F25902dC2761A0d4CA6
Beneficiary    : 0x1bCE6d61F2cFE40F0879b17a43780695CbCc19ff
Node           : 0x0C5393db793DbA88f16DC4D030D678FBD88F8B0D
Start          : 1 778 857 909
Duration       : 14 400 s   (4 h)
Cliff          : 0
Funded         : 100 IDOS
Stake tx       : 0xf0364627274f56b6e4929cb2252ad6a6032fac5480d891d8d02d128d475131ef
Release tx     : 0x78378500fed7858bff26ab9f3ba3024214728df5e722a018a6050e1ab79cb5d4
Unstake tx     : 0x7e53423e3b432864678d14d40aa2075fd650c7c50553bf56a4b6d6da6e849b71
Premature withdraw : reverted with NoWithdrawableStake (selector 0xf395c842)
```

— end of report —